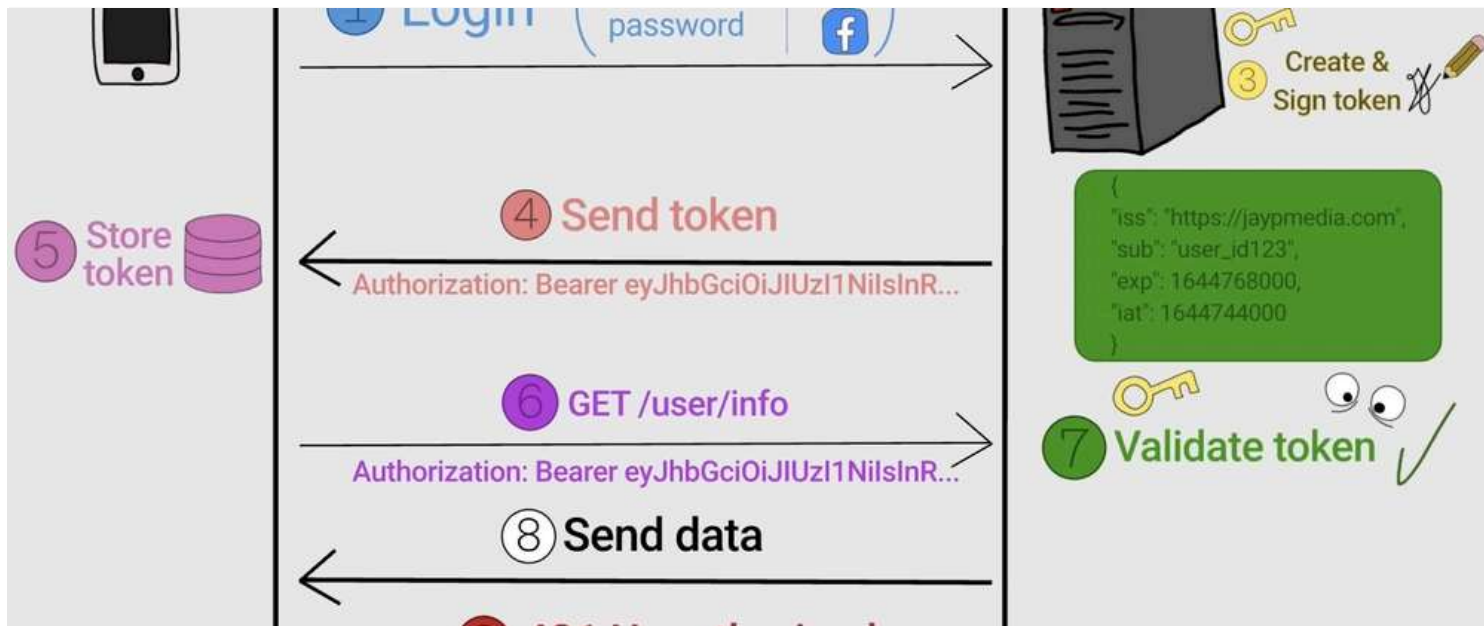




Jean-Paul Rustom

Posted on Feb 24, 2024 • Edited on Mar 16, 2024



239



7



7



8



8

JWT explained in 4 minutes (With Visuals)

#webdev #javascript #tls #node

JWT - JSON Web Token explained in 4 minutes (With Visuals)



Introduction

JWT authentication and session authentication are ways to authenticate users of your web app.

In this article we will explain the details of JWT, its structure along with its pros and cons. JWT stands for **JSON Web Token**, and it is a commonly used stateless user authentication [standard](#) used to securely transmit information between client and server in a JSON format.

A JWT is encoded and not encrypted by default.

It is digitally signed using a secret key known only to the server.

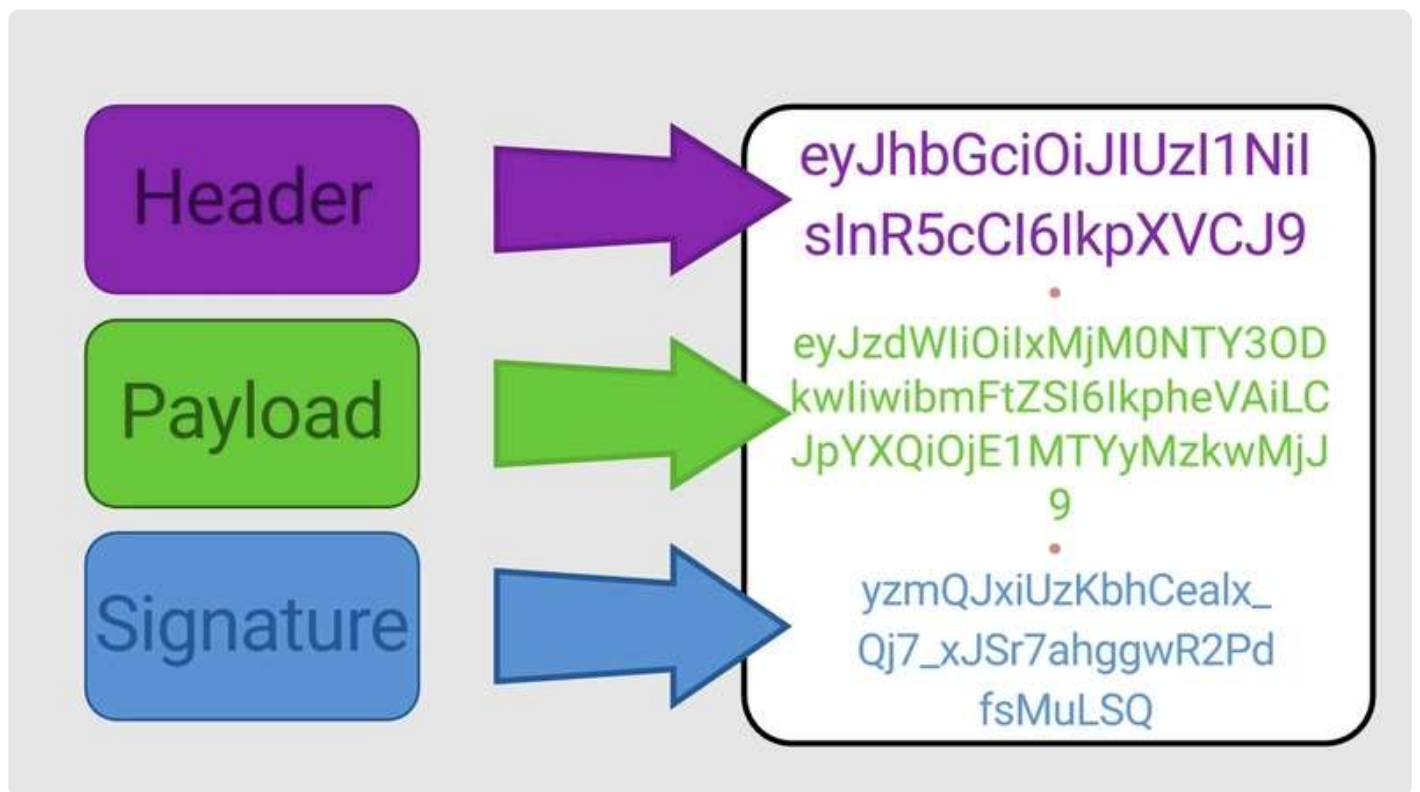
It can be encrypted, but for this article we will focus on signed non-encrypted tokens.

Structure of JWT

A JSON Web Token consists of 3 parts separated by a period.

The header, the payload, and the signature.

Each section is base64 encoded.



Header

The header consists of token type, which is JWT, and the signing algorithm used, such as HMAC SHA256 or RSA.

```
{  
  "typ": "JWT",  
  "alg": "HS256"  
}
```

Payload

The payload consists of the claims.

Claims are statements about the user, and additional data.

For example, we have the time the token was issued at.

We also have its expiration time, because tokens should expire.

```
{
```

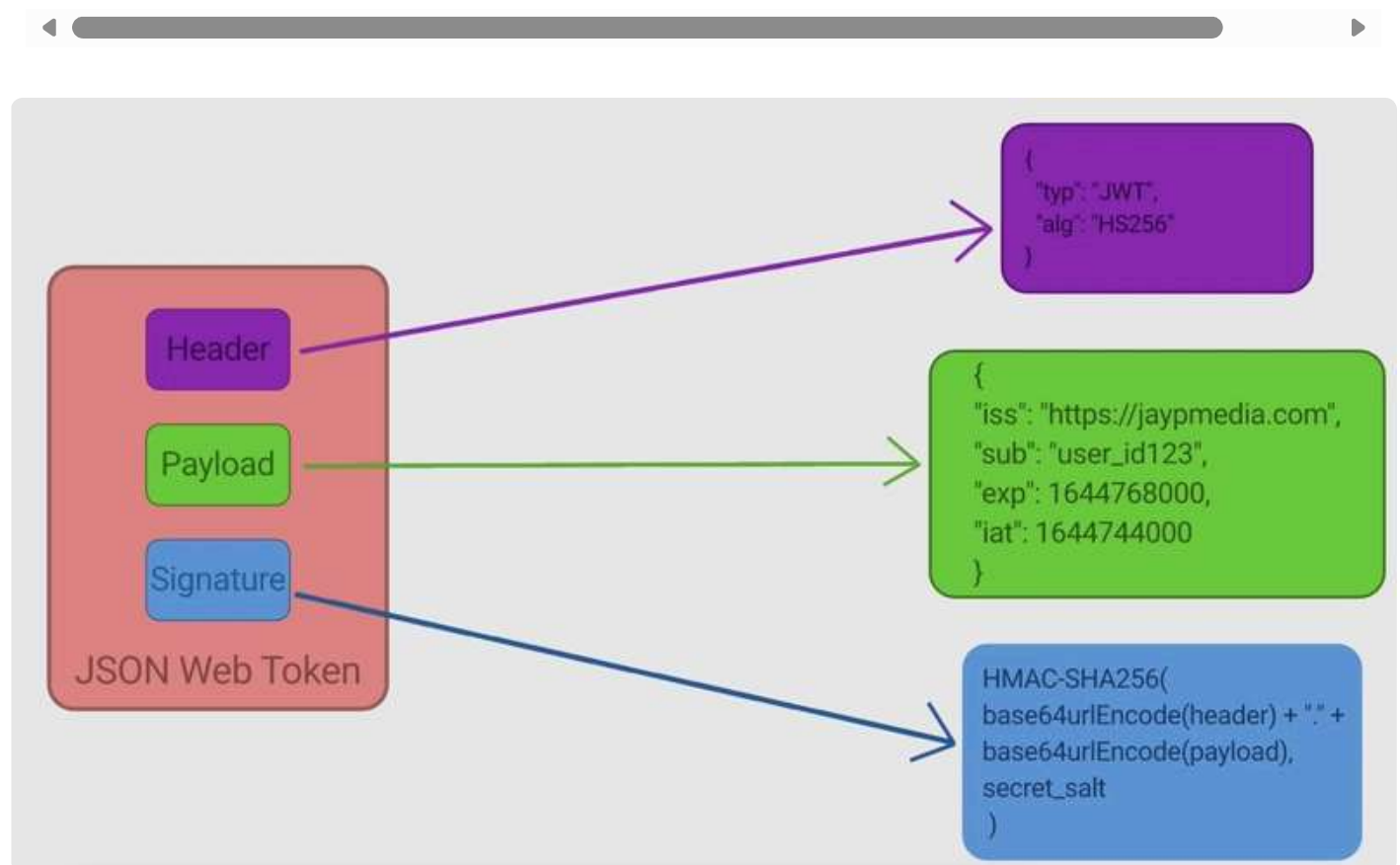
```
"iss": "example_issuer",
"sub": "user_id123",
"exp": 1644768000,
"iat": 1644744000
}
```

Signature

The signature is most important part of a JWT.

It is calculated using the header, the payload, and the secret, which are fed to the signing algorithm to use.

```
signature = HMAC-SHA256(base64urlEncode(header) + "." + base64urlEncode(payload), secret)
```



Steps

The steps involved in a typical JWT authorization flow are as follows:

1- Authentication: The user signs in using username and password, or using for example Google or Facebook.

The server verifies the provided credentials.

2- Token Generation & sending token to client: The server will generate the JWT and send it to the client, which stores it for future use.

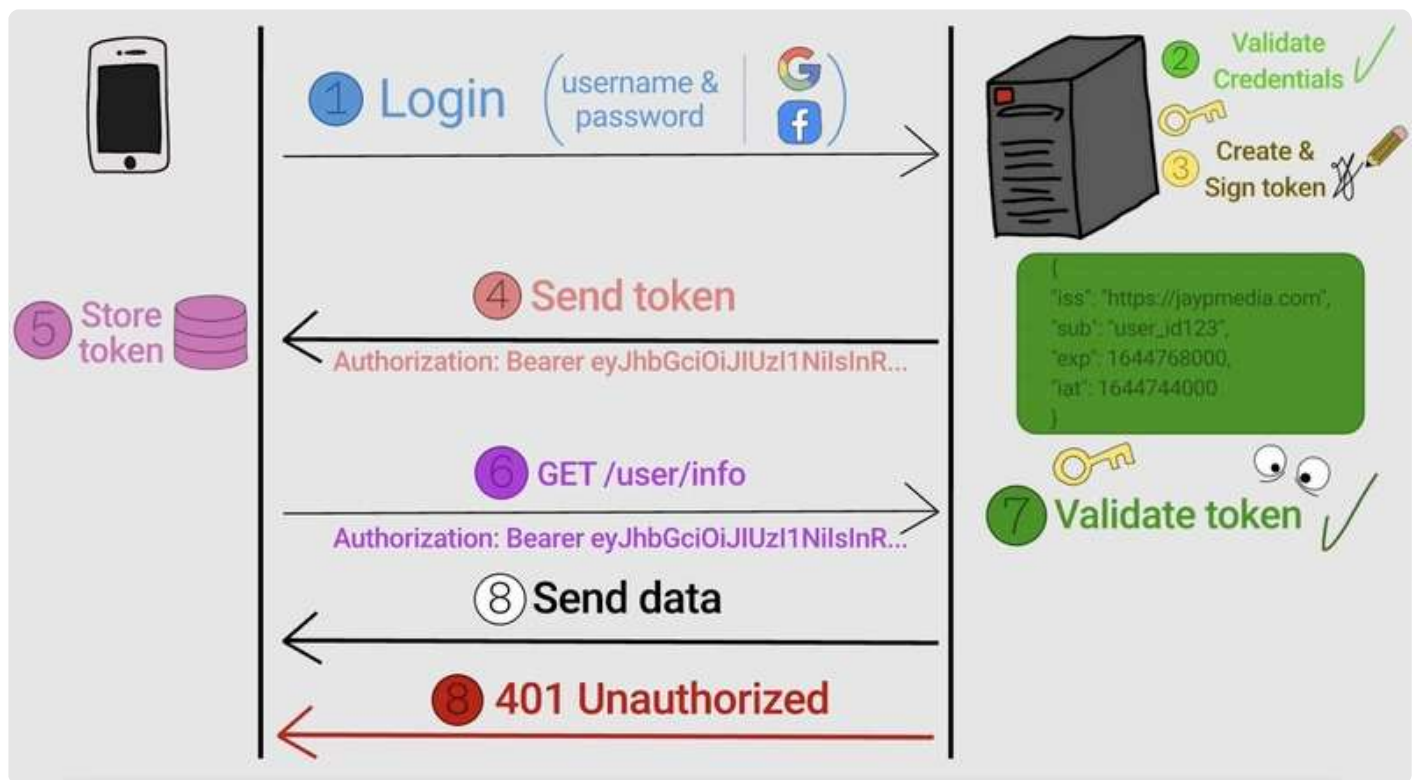
3-Sending the Token to server: When the client wants to access a protected resource on the server, it sends the JWT in the Authorization header of the HTTP request.

```
axios.get(URL, {  
  headers: {  
    'Authorization': 'Bearer ' + token,  
  },  
})
```

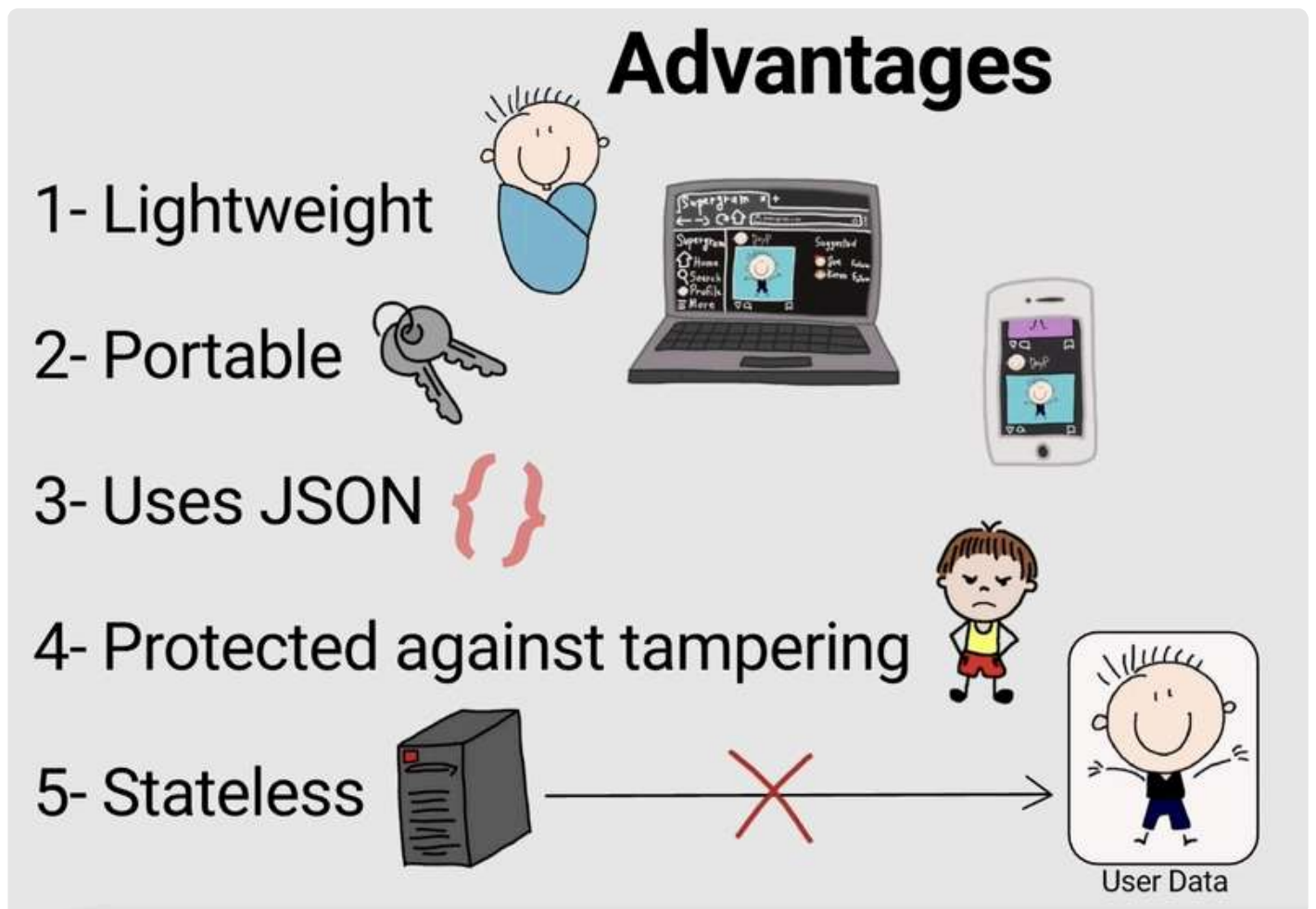
4-Verifying the Token: The server receives the request and verifies the JWT by checking its signature using the secret key that was used to sign it.

If the JWT is valid, the server extracts the information contained in it and uses it to determine what actions the user is authorized to perform.

5- Authorizing the Request: If the user is authorized to access the resource, the server returns the requested data. If the user is not authorized, the server returns an error message.



Advantages



- Lightweight
- Portable: can be processed on multiple platforms, web and mobile.
- JSON parsers are common in most programming languages.
- Protected against tampering because of the secret key stored on server side.
- The server does not need to store any session information, because of the stateless nature of the JWT token.

Disadvantages

Disadvantages

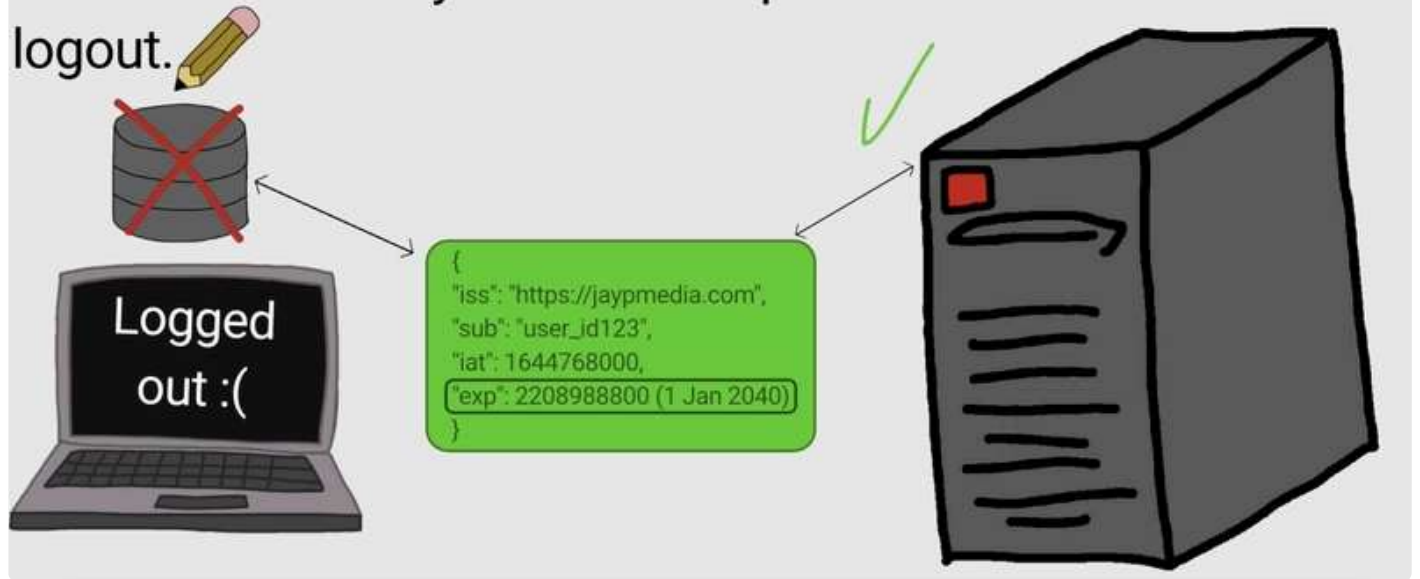
1- Should manually mark non-expired JWT as invalid on logout. 

2- Should not store sensitive info 

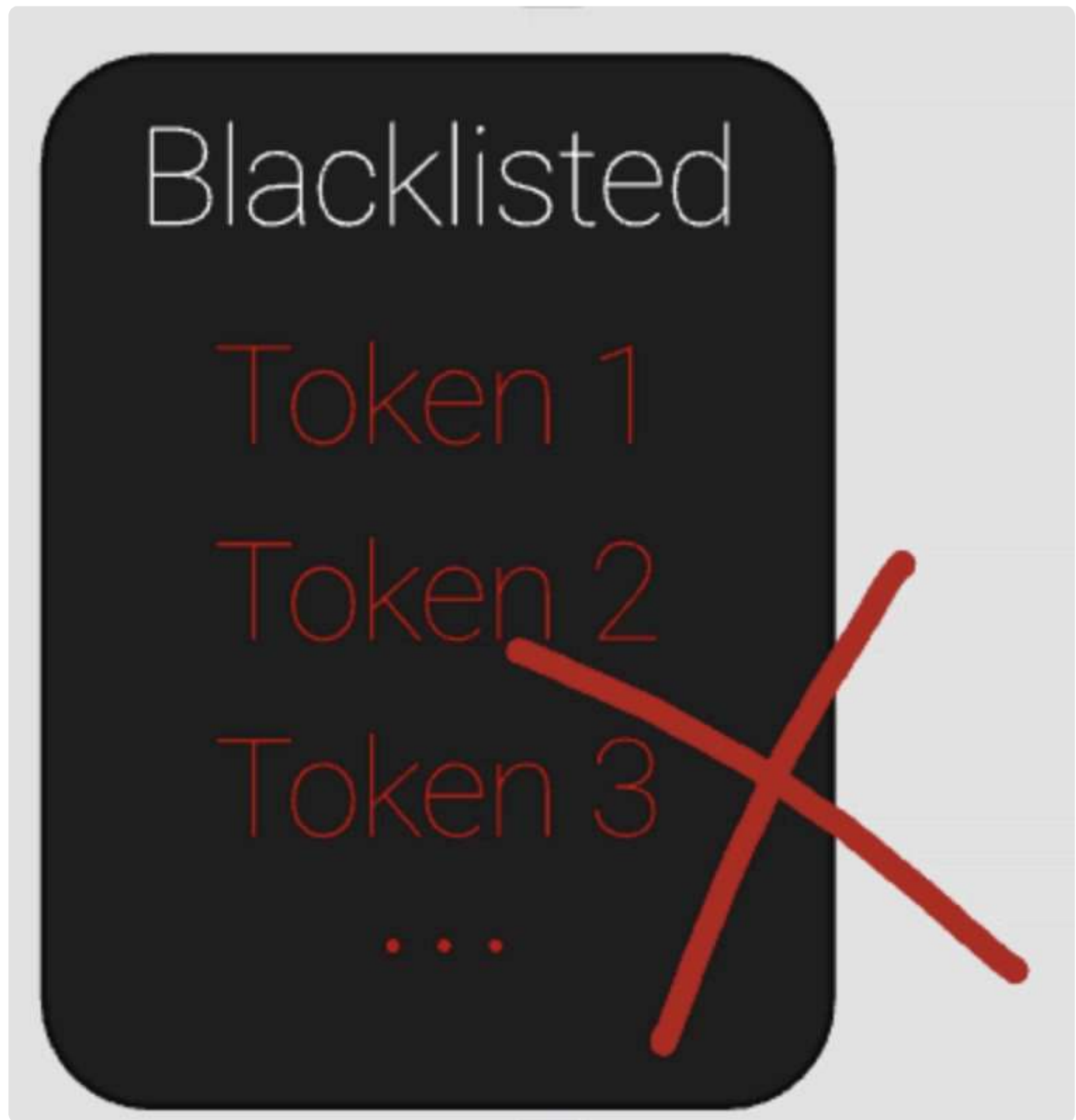
3- On client side should be stored somewhere secure 

- On server side, should manually mark non-expired JWT as invalid on logout. A JWT can still be valid even after it has been deleted from the client.

1- Should manually mark non-expired JWT as invalid on logout.



A method used is token blacklisting, which involves maintaining a list of invalidated tokens on the server side, preventing their reuse for authentication.



- For signed non encrypted tokens, we should not store sensitive info because JWT is protected against tampering but can be decoded and read by anyone.
- Can provide full access if intercepted. That's why JWTs on client side should be stored somewhere secure, for example in the browser in an HttpOnly cookie.

My other articles:

- [OAuth 2.0 Explained In Simple Terms \(With Visuals\)](#).
- [What happens when you click a URL - DNS Lookup, TCP Handshake & HTTP \(With visuals\)](#).
- [TLS Certificates !\[\]\(efb87da6d8ca3116acedf2a9895074d9_img.jpg\) - Simplified for web developers](#)

OAuth 2.0 Explained In Simple Terms (With Visuals)



WebSockets Explained Under 10 Minutes (With Visuals)



What happens when you click a URL: DNS Lookup, TCP Handshake & HTTP I...



Top comments (12)



nigel447 • Feb 25 '24 • Edited



"JWT is protected against tampering" true to a limited extent

scenario 1:

the app gets a token from the server, but a proxy(like burp) intercepts the token then

decodes the payload, alters parameter(s) for example some user id parameter like email for example then passes the token to the app, since the app more than likely does not check the signature(usually verify is on the back-end) the app will now make a state update on altered user data

scenario 2:

there is a bunch of [attacks based on the "algo" parameter](#)

strongly advise working on the premise the JWT has been tampered with and coding defensively against this, for example scenario 1, resend the JWT back to the back-end for verification before app state update

Misha • Mar 3 '24



You described MITM attack where malicious proxy can read and modify arbitrary data between client and server. If client sends back to the server jwt token for verification, what prevents proxy from intercepting this request and spoofing the response?

nigel447 • Mar 3 '24 • Edited



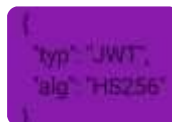
u are correct nothing stops this, hopefully you can see that just trusting that the jwt is valid is an error, basic idea is if you get a jwt that does not verify on the server then this is a red flag that you are under attack and you then implement defensive code, which is better than just hoping everything is ok, as to spoofing the response need correct [headers](#)

Schemetastic (Rodrigo) • Feb 25 '24 • Edited



Hey cool video! I did learn new stuff and it was very interesting!

If you let me give you a bit of feedback, in some parts the contrast of some text wasn't so good, let me show some examples:



Maybe bolder text or more light colors, keep in mind that I watched most of the video in a small size (not full screen). Hope this helps.

But it was a very cool video!

Sudipta Adhikary Joy • Mar 17 '24



Great content to learn about JWT. I need more content like this. Appreciate it and Thank you providing such a beautiful content.



Jose Galdamez • Mar 2 '24



✕ JWT is one of those subjects I keep having to re-learning every few years since I don't need to implement it often. When it's already working you don't have to worry about it. Overall, great video animation and write-up! It definitely beats reading the standard for getting a quick summary.



Thomas Broyer • Feb 27 '24



✕ JWTs are very common (too common?), but have also been harshly criticized by security professionals; see end of:



tbr

What are JWT?

over

Thomas Broyer • Nov 29 '23

#jwt #security #webdev



harkinj • Feb 28 '24



✕ Hi,
'4-Verifying the Token'. - does this involve calling the 'auth server'?
Thanks



Jakub Serafin • Mar 5 '24



✕ this is important step that is totally missing in this article.



Marco Palamede • Apr 15 '24



✕ Very nice article. Do you have it in PDF or eBook ?



Ricardo Esteves • Feb 28 '24





Nice article, thanks for sharing !

[View full discussion \(12 comments\)](#)

Some comments may only be visible to logged-in visitors. [Sign in](#) to view all comments.

[Code of Conduct](#) • [Report abuse](#)

EDUCATION

Bachelor of Computer Science

PRONOUNS

him

WORK

Software Developer

JOINED

Oct 14, 2020

More from [Jean-Paul Rustom](#)

OAuth 2.0 Explained In Simple Terms (With Visuals)

[#webdev](#) [#javascript](#) [#node](#) [#mobile](#)

WebSockets Explained Under 10 Minutes (With Visuals)

[#webdev](#) [#backenddevelopment](#) [#javascript](#) [#mobile](#)

What happens when you click a URL - DNS Lookup, TCP Handshake & HTTP (With visuals)

[#webdev](#) [#javascript](#) [#tls](#) [#node](#)